

Einführung in den systematischen Softwaretest

Modul II2005 im BSc Informatik der THM

Andreas Gareiß

Fachbereich Mathematik, Naturwissenschaften und Informatik

21. Oktober 2025



CAMPUS
GIESSEN

MNI

Mathematik, Naturwissenschaften
und Informatik

Einführung

Die Ursachen von Softwarefehlern sind oft sehr komplex!

Kommunikationsprobleme Die Entwickler wussten nicht genau, was sie tun sollten oder die Spezifikation war ungenau oder Auftraggeber und Entwickler haben sie unterschiedlich verstanden.

Psychologische oder kulturelle Probleme Entwickler wollten Unklarheiten nicht zugeben.

Falsche Erwartungen Das Marketing hat Funktionen und Eigenschaften zugesichert, die nicht spezifiziert waren und nicht implementiert wurden.

Software ist immateriell Im Gegensatz zu Industrieprodukten ist Software nicht greifbar. Auch eine optische Prüfung (Review des Quellcodes) ist sehr aufwändig.

Geänderte Anforderungen Wurden Teile der Software vor der Änderung geschrieben?

Komplexität der Software In großen Projekten: Kein Entwickler versteht mehr die ganze Software.

Schnittstellenprobleme Zwischen eigenen Modulen oder zwischen eigenen und fremden Modulen.

Zeitdruck Oft nach falscher Aufwandsschätzung, zieht oft Fehler nach sich (Entwickler unter Druck) und dann fehlt auch noch die Zeit zum Testen.

Wiederverwendung von Code Der wiederverwendete Code funktioniert evtl. anders als erwartet, wurde evtl. unter ganz anderen Bedingungen getestet, war vielleicht schlecht dokumentiert.

Menschliche Fehler Entwickler sind keine Roboter und machen manchmal einfache, dumme Fehler.

Ungetestete Programmteile Die Kunden gehen anders als die Entwickler mit dem Programm um, führen andere Situationen herbei.

Andere Ausführungsreihenfolge Entwickler haben nicht alle möglichen Reihenfolgen probiert.

Andere Eingangsdaten Beim Kunden werden Kombinationen von Eingangsdaten verarbeitet, die der Entwickler nicht getestet hat. Die Zahl der Kombinationen kann sehr groß sein, nicht alle werden getestet.

Größere Datenmenge Im Ernstfall werden viel größere Datenmengen verarbeitet, als die Entwickler zum Testen benutzt haben.

Keine Daten Es wurde immer mit vorhandenen Daten entwickelt. Nun sind beim Start des Projektes aber keine Daten existent.

Andere Umgebung Beim Kunden ist das Betriebssystem, Netzwerk, Dienste, Storage ... anders als beim Entwickler [Grü17]

Steigender Umfang an SW Die Menge an Software ist insgesamt stark wachsend, die Kunden verlangen mehr Komfort, schönere Oberflächen, mehr Kommunikationsmöglichkeiten. Folge: mehr Komplexität.

Neue Anwendungsbereiche Software wird heute eingesetzt, wo früher mechanische und elektrische Systeme zum Einsatz kamen, Vordringen der Embedded Systems, SW läuft in Autos, Flugzeugen, Satelliten, Robotern. Folge: Neue Fehlermöglichkeiten.[Grü17]

Restfehlerraten

Fachleute gehen von folgenden Zahlen aus:

- 25–50 Softwarefehlern pro 1000 Lines of Code (KLOC)
- Nach dem Testen verbleiben 1–3 Restfehler pro KLOC
- Davon ist ca. jeder 10. Fehler schwerwiegend.

Das ergibt rechnerisch die folgende Anzahl Restfehler bei typischen Applikationen:

Applikation (Software)	Umfang SW (LOC)	anfängl. Fehler	Restfehler nach Test	Schwerwiegende Restfehler
Raketen- steuerung	30.000	1.500	60	6
Navigation Spaceshuttle	500.000	25.000	1.000	100
Flugkontrolle Europa / USA	1.000.000	50.000	2.000	200
Steuerung Kernkraftwerk	1.500.000	75.000	3.000	300
Linux Kernel 4.2	20.200.000	1.010.000	40.400	4.040
Facebook	61.000.000	3.050.000	122.000	12.200
Google Services	2.000.000.000	100.000.000	4.000.000	400.000

Beispiel: Venus-Sonde

Die Venussonde Mariner 1 geht 1962 verloren. Ursache: Ein Fehler im Programmcode. In dem Fortran-Programm war an einer Stelle ein Punkt statt eines Kommas. Schaden: 80 Mill. \$ (andere Version: Ein Bindestrich fehlte)

Beispiel: Mars Orbiter

Die Raumsonde Mars Orbiter der NASA sollte 1999 auf dem Mars landen. Sie funktionierte während des ganzen Fluges problemlos, ging aber kurz vor der Landung verloren. Es stellte sich heraus, dass ein Fehler in der Software der Bodenstation war. Es handelte sich um die Software, die den Schub für die nötigen Kurskorrekturen berechnet. Ein Teil der Software hatte den Schub in Newton berechnet, ein anderer Teil (von einem anderen Team) die Einheit Pounds verwendet. Schaden: 125 Mill.\$

Beispiel: Mars Sojourner

Das autonome Erkundungsfahrzeug Sojourner wurde 1997 auf dem Mars abgesetzt und sollte den Planeten erkunden. Schon im ersten Monat wurden 1.2 GByte Daten (Messdaten und Fotos) zur Erde gefunkt. Nach ca. 3 Monaten brach die Verbindung zur Erde kurz ab, Sojourner ging nun durch häufige Resets und funkte viel weniger Daten. In der Software (unter Echtzeitbetriebssystem VxWorks) gab es eine Prioritäten-Inversion. Ein Task niedriger Priorität blockierte Ressourcen und dadurch kam ein Task höherer Priorität nicht zum Zug und der Watchdog-Timer wurde ausgelöst. Der Grund lag in der unerwartet großen Datenflut, ein solcher Umfang war nicht getestet worden.

Beispiel: Ariane 1/2

Bei Ihrem Jungfernflug 1996 wurde die Ariane 5 nur 36.7 Sekunden nach dem Start gesprengt. Es stellt sich heraus, dass ein Unterprogramm der Steuerungssoftware eine Kursabweichung gemeldet hatte, die gar nicht vorlag. Die Ariane versuchte nun eine extrem harte Kurskorrektur und begann auseinander zu brechen, daraufhin wurde das Neutralisationsprogramm aktiv und sprengte die Rakete. Folgende Ursache wurde ermittelt: In der Software entstand bei der Übermittlung der Horizontalgeschwindigkeit. Eine Umwandlung einer Gleitkommazahl in eine signed 16-Bit-Zahl ein Zahlenüberlauf, die Werte waren größer als 32767. Dadurch entstanden völlig unsinnige Werte. Der Backup-Bordcomputer hatte die gleiche Software und stieg kurz nach dem Hauptcomputer aus.

Beispiel: Ariane 2/2

Die betroffene Software war von der Vorgängerrakete Ariane 4 übernommen worden, wo solche großen Horizontalgeschwindigkeiten nicht erreicht wurden. Die Routine wurde bei der Übernahme in die Ariane 5 nicht erneut getestet. Tragisch war, dass diese Software nur 40 s mitlaufen sollte, um einen Startabbruch besser managen zu können. Dieser Teil der Software hätte aber nur am Boden mitlaufen müssen. Schaden: 370 Mill.\$

Beispiel: Patriot-Rakete 1/2

Im Golfkrieg 1991 verfehlte eine Patriot-Abwehrrakete eine angreifende irakische Scud-Rakete. Diese flog weiter und traf eine Kaserne, 28 Menschen (US-Soldaten) kamen ums Leben. Es stellte sich heraus, dass die Software in der Patriot fehlerhaft war und mit der internen Zeitberechnung zu tun hatte.

Beispiel: Patriot-Rakete 2/2

Die interne Zeit wurde ganzzahlig in Zehntelsekunden geführt. Um in Sekunden umzurechnen wurde mit $1/10$ mal genommen, was schneller geht als eine Division. Der Umrechnungsfaktor $1/10$ wurde als Festkommazahl dargestellt:

$$U = \frac{1}{2^4} + \frac{1}{2^5} + \frac{1}{2^8} + \frac{1}{2^9} + \frac{1}{2^{12}} + \frac{1}{2^{13}} + \dots$$

Bei dieser Darstellung wurden 24 Bit verwendet und es verblieb ein Restfehler von $\approx 0.95 * 10^{-7}$. Nach 100 Betriebsstunden ergibt sich damit eine Zeitdifferenz von

$$\Delta t = 0.95 * 10^{-7} * 100 * 3600 = 0.34s$$

In diesen 0.34s fliegt die angreifende Scud-Rakete 570m und wurde deshalb nicht mehr von der Abwehrrakete gefunden.

Beispiel: F16-Flugzeug #1

Das bei General Dynamics hergestellte Kampfflugzeug F-16 ist ein fly-by-wire-aircraft. Dabei werden die Steuerbewegungen des Piloten registriert und an einen Computer weiter gegeben. Dieser prüft ob das Flugmanöver in Ordnung ist und steuert das Flugzeug entsprechend. Bei einer Simulation wurde festgestellt, dass das Flugzeug sich auf den Rücken dreht, wenn der Äquator überflogen wird. Angeblich stammt der ursächliche Teil der Software aus einer Raketensteuerung, wo dieses Verhalten toleriert wurde.

Beispiel: Frühphase F16-Flugzeug #2

Bei einer auf dem Rollfeld abgestellten F-16 wurde aus Neugier der Computer angewiesen das Fahrwerk einzuziehen. Er tat es und die F-16 war defekt.

Beispiel: Frühphase F16-Flugzeug #3

Bei der F-16 können Tanks und Bomben abgeworfen werden. (Der Computer entscheidet jeweils welche Bombe oder welcher Tank abgeworfen wird, um das Flugzeug ausbalanciert zu halten). Es stellte sich heraus, dass dieser Mechanismus auch im Rückenflug funktionierte.

Beispiel: Alarm des sowjetischen Raketenfrühwarnsystems

Am 26.9.1983 meldete das sowjetische Raketenfrühwarnsystems eine Angriff einer amerikanischen Interkontinentalrakete. Kurz darauf meldete das System vier weitere Raketen. Glücklicherweise stufte der diensthabende Offizier (Oberstleutnant Petrow) das Ganze als Computerfehler ein und leitete nicht den nuklearen Gegenschlag ein. Damit hat er möglicherweise einen dritten Weltkrieg verhindert.

Es stellte sich heraus, dass das satellitengestützte Frühwarnsystem Sonnenreflexionen auf Wolken in Montana fälschlich als startende Raketen erkannt hatte. (Wikipedia)

Beispiel: Unfall mit dem A320

Im Jahre 1993 fuhr ein A320 der Lufthansa nach der Landung in Warschau über das Ende der Landebahn hinaus und krachte in einen Erdwall. Dabei kamen zwei Menschen ums Leben. Der Bremsweg war so groß, weil der Umkehrschub erst sehr spät einsetzte. Die Untersuchung ergab, dass die Software den Umkehrschub erst frei gibt, wenn alle Stoßdämpfer des Fahrwerks mit 12 Tonnen Kraft zusammengepresst werden.

(spezifikationsgemäß) Wegen starkem Seitenwind lag bei der Landung die Maschine schräg, ein Rad setzte sehr spät und nicht mit voller Kraft auf.

Die Lufthansa wies nun sofort ihre Piloten an, bei Seitenwind besonders hart aufzusetzen, bis in Absprache mit Airbus die Spezifikation geändert wurde. Nun kann der Umkehrschub schon bei 2 Tonnen pro Stoßdämpfer einsetzen.

Beispiel: Absturz eines Lockheed F-117 Tarnkappenbombers

Ein trivialer Fehler führte 1982 zum Absturz eines Lockheed F-117 Tarnkappenbombers: Die Steuerung des Höhenruders wurde mit dem Seitenruder vertauscht.

Beispiel: Eisenbahn-Stellwerk Hamburg Altona

1995 wird ein älteres Stellwerk mit 50 Mitarbeitern ersetzt durch eine computergestützte Anlage mit 486-Rechner und Echtzeitbetriebssystem. Nach Inbetriebnahme stürzt die Software ab und das Stellwerk muss geschlossen werden. Die Folge ist ein Chaos im Bahnverkehr mit deutschlandweiten Auswirkungen.

Ursache: Die Software versuchte bei starkem Zugverkehr einen Stack mit 4 kB anzulegen. Nötig sind eigentlich nur 3.5 kB aber der Programmierer wollte „sicherheitshalber“ etwas mehr anlegen. Leider waren hardwareseitig nur 3.5 kB eingebaut.

Beispiel: Gepäcksortierung am Flughafen Denver

Nach der Neueröffnung des Flughafens Denver soll ein vollautomatisches Gepäcksystem arbeiten. Dieses arbeitet fehlerhaft, es gibt zerquetschte und verlorene Gepäckstücke. Das Gepäck muss wieder mit der Hand sortiert werden, Flughafen wird 16 Monate verspätet eröffnet, Schaden 3.2 Milliarden \$.

Ursache war eine Überlastung des LAN (Ethernet), die große Anzahl an Nachrichten konnte nicht verarbeitet werden. Komplexität wurde unterschätzt.

Beispiel: Softwarefehler im Kassensystem von Lidl

Im Jahr 2008 stellte sich heraus, dass im Kassensystem von Lidl ein Softwarefehler schlummerte. Jedes mal, wenn die Kassensumme exakt gleich Null war, stürzte die Kassensoftware ab. Diese Null konnte bei Abgabe von Leergut leicht erreicht werden und viele Spaßvögel führten diesen Absturz mit Absicht herbei um die verdutzten Gesichter des Personals zu sehen.

Beispiel: Bankensoftware

Die Bank of America hat 1988 große Computerprobleme, vorübergehender Verlust von mehreren Millionen Dollar, viele Kunden gehen weg.

Die US Federal Reserve (Zentralbank) nimmt ein neues Computersystem in Betrieb, 28 Milliarden werden irrtümlich überwiesen, 4 Milliarden kommen nicht zurück.

Beispiel: Jahr-2000-Bugs

Obwohl der größte Teil der Software gut ins Jahr 2000 kam, gab es doch vereinzelt Probleme. So erhielten Autobesitzer in Maine Benachrichtigungen auf denen ihre Autos als „Kutschen ohne Pferde “bezeichnet wurden. Grund: Das Jahr 2000 wurde zweistellig dargestellt, also „00 “und die „19 “automatisch davor gesetzt. Ergibt: „1900 “.

Beispiel: Bestrahlungsgerät

In den 80er Jahren gab es in den USA und Kanada einige Unfälle mit einem Bestrahlungsgerät namens Therac-25. Dabei erhielten einige Patienten eine Strahlungs-dosis die mehr als das 100-fache der üblichen Dosen betrug. Die meisten davon starben an den Folgen der Überdosis. Es stellte sich folgende Ursache heraus: Eine schwer entdeckbare Race Condition in Verbindung mit falschen bzw. nichtssagenden Fehlermeldungen.

Beispiel: Automobil

Es gibt es der Automobilbranche zahlreiche Beispiele für Fehlfunktionen die durch Softwarefehler verursacht wurden. Beispiele sind nicht aufgehende oder willkürlich aufgehende Scheiben, falsch anzeigende Instrumente u.v.m.

Eines der neuesten ist der Autopilot von Tesla. Dieser hielt einen querenden LKW für eine Schilderbrücke. Der Fahrer kam ums Leben.

„Bei diesem Unfall führte die hohe weiße Seitenwand des Anhängers zusammen mit der Radar-Signatur, die der eines hoch hängenden Straßenschilds sehr ähnlich war, dazu, dass keine automatische Bremsung ausgelöst wurde.“

Beispiel: Frauen zählen im Flugzeug als Kinder?

Im Juli 2020 sind drei Flugzeuge von TUI Airways mit falsch berechnetem Gesamtgewicht und daraus resultierend etwas zu wenig Schub gestartet. Die für die Untersuchung von Flugunfällen zuständige Abteilung Air Accidents Investigation Branch (AAIB) der britischen Verkehrskontrolle spricht in ihrem Bericht von einem »ernsten Zwischenfall«.

Demnach habe ein Programmierfehler dafür gesorgt, dass erwachsene Frauen vom Check-in-System unter Umständen als Kinder registriert wurden – und zwar immer dann, wenn sie als Anrede »Miss« ausgewählt hatten. Der Fehler wurde im Rahmen eines Upgrade-Prozesses der Check-in-Software eingeführt.

Beispiel: Frauen zählen im Flugzeug als Kinder?

»Das System wurde nicht in Großbritannien programmiert. Dort, wo die Programmierung durchgeführt wurde, wird Miss für ein Kind verwendet und Ms für eine erwachsene Frau, daher der Fehler«, heißt es im AAIB-Bericht.

Die Folge: Allen Passagierinnen, die am Flughafen als Miss eincheckten, wurde ein Durchschnittsgewicht von 35 kg zugerechnet, statt der 69 kg. Auf einem Flug von Birmingham nach Palma de Mallorca führte das zu einer Abweichung von 1244 Kilogramm, sprich: Das Flugzeug wog 1,2 Tonnen mehr, als den Piloten gemeldet wurde. Denen fiel vor dem Start zwar noch auf, dass das ursprünglich vorhergesagte Gesamtgewicht nicht mit dem letztlich nach dem Check-in gemeldeten übereinstimmte und dass offenbar unerwartet viele Kinder an Bord waren, nämlich 65 statt der vorhergesagten 29, sie hielten das aber für plausibel.

Beispiel: Frauen zählen im Flugzeug als Kinder?

Ein späterer Abgleich zeigte, dass das Flugzeug im Optimalfall mehr Schub und eine etwas höhere Startgeschwindigkeit gebraucht hätte, als es aufgrund der Fehlberechnung letztlich hatte. Da der Schub aber aufgrund der Wetterbedingungen gerade noch den regulatorischen Anforderungen entsprach, sei »der sichere Betrieb des Flugzeugs nicht kompromittiert« gewesen.

Beispiel: Das Jahr-2038-Problem – Y2K38

Das nächste Software-Problem wartet schon auf uns: Im Jahr 2038. Denn am 19. Januar um 03:14 Uhr und 8 Sekunden könnte es einem Problem kommen, welches alle bisher dagewesenen Software-Bugs in den Schatten stellt. Es betrifft so gut wie alle Computer.

Überblick

- Todesfälle: **81**
- Lebensgefahr: **286**
- Verlust von Sachwerten: **476**
- Andere Risiken: **319**

Es gibt mit Sicherheit eine hohe Dunkelziffer!

Übung

Gehen Sie die geschilderten Folgen bekannt gewordener Softwarefehler durch und schreiben Sie – so weit bekannt – jeweils auf, wie die Fehlerursache in die oben genannten Gruppen einzuordnen ist.

Haftung des Softwareherstellers

Der Gesetzgeber erwartet die Befolgung des Stands der Technik und der anwendbaren Normen. Alle Schritte einschließlich der Tests müssen sorgfältig dokumentiert werden.

Auch die Entwickler sind zur Sorgfalt verpflichtet, besonders bei sicherheitsrelevanten Systemen. Bei grober Fahrlässigkeit oder Vorsatz kann ein Entwickler persönlich haften.

Murphy's Gesetz

Die Ingenieure haben gelernt, die Dinge mit schwarzem Humor zu nehmen. Berühmt wurde Murphy's Law:

„Anything that can go wrong, will go wrong“

Murphy's Gesetze für Informatiker 1/2

Die Informatiker haben ihre eigenen Versionen von Murphy's Gesetz:

- Jedes fertige Programm, das läuft, ist veraltet.
- Wenn ein Programm nützlich ist, muss es geändert werden.
- Wenn ein Programm nutzlos ist, muss es dokumentiert werden.
- Jede Programmentwicklung wächst so lange, bis es die Fähigkeiten des Programmierers übertrifft, der es weiterführen muss.
- Erst wenn ein Programm mindestens 6 Monate läuft, wird der katastrophale Fehler entdeckt werden.

Murphy's Gesetze für Informatiker 2/2

Die Informatiker haben ihre eigenen Versionen von Murphys Gesetz:

- Ein sorglos geplantes Projekt dauert viermal so lange als erwartet.
Ein sorgfältig geplantes Projekt nur zweimal so lange.
- Jeder Programmierer programmiert stets so lange bis der ihm zur Verfügung stehende Speicher restlos aufgebraucht ist.
- Der Fehler befindet sich immer in der Routine, die niemals getestet wurde.
- Planung ersetzt Zufall durch Irrtum.

Literatur